



National University of Sciences and Technology (NUST)
School of Electrical Engineering and Computer Science

Department of Computing

School of Electrical Engineering and Computer Science

CS-250: Data Structure and Algorithms

Class: BSAI 2A

**Lab 1: Revisiting C++, Introduction to Version Control & Writing Test Cases
in C++**

Date: 09th September 2026

Time: 10:00 am – 12:50 pm

Instructor: Saud Kamran

Lab Engineer: Urooj Akmal



Lab 1: Revisiting C++, Introduction to Version Control & Writing Test Cases in C++

Introduction

This lab is about revising C++ programming concepts through moderately challenging problems that serve as a foundation for Data Structures and Algorithms (DSA). Students will also practice using GitHub and version control to manage their code. In C++, arrays, strings, and matrices are fundamental structures upon which advanced data structures are built. Students will implement modular functions in C++ and verify their correctness through systematic test cases.

Objectives

- Students will practice basic Git commands and use GitHub for version control.
- Students will revise and strengthen programming concepts learned in earlier courses.
- Students will practice modular programming by writing reusable C++ functions.
- Students will write systematic software tests for each implemented function.
- Students will prepare for advanced data structures by solving moderately difficult problems.

Tools/Software Requirement

- Visual Studio C++ (or any standard C++ IDE such as Code::Blocks, CLion, or g++ compiler).
- GitHub for version control and submission

Description

Students will begin by revising version control concepts using GitHub, followed by a series of C++ programming tasks. Each task emphasizes correct coding style, modularity, and testing. Students will commit their solutions to GitHub and submit their completed work on LMS in Word format.

Lab Tasks

Task 1 – GitHub Repository Setup (Step-by-Step)

Students will create and prepare a GitHub repository for this lab.

1. Create a new repository on GitHub

- Repository name: DSA-Lab1
- Visibility: Private or Public (as instructed).
- Initialize with: README.md
- Add a license: (optional) MIT



- Add .gitignore: C++ (students may refine later)

2. Clone the repository to a local machine

```
git clone https://github.com/<your-username>/DSA-Lab1.git
cd DSA-Lab1
GitHub Link
```

3. Set Git identity (first time only on a machine)

```
git config --global user.name "Your Name"
git config --global user.email youremail@example.com
pull.rebase=false
credential.helper=manager
credential.https://dev.azure.com.usehttppath=true
init.defaultbranch=master
user.name=River
user.email=nrg0773@gmail.com
core.repositoryformatversion=0
```

4. Verify the remote

```
git remote -v
PS D:\DSA\Lab1> git remote -v
origin https://github.com/nrg0773-dotcom/DSA-Lab1.git (fetch)
origin https://github.com/nrg0773-dotcom/DSA-Lab1.git (push)
PS D:\DSA\Lab1>
```

5. Create a simple C++ starter file

```
#include <iostream>
using namespace std;

int main() {
    cout << "Welcome to DSA Lab 1!" << endl;
    return 0;
}
```

6. Build/run locally (example using g++)

```
g++ hello.cpp -o hello
./hello
```

7. Stage, commit, and push

```
git status
git add hello.cpp
git commit -m "Add hello.cpp starter program"
git push origin main # If your default branch is master, use 'master'
```



DSA-Lab1 Public

master 1 Branch 0 Tags

Go to file T Add file <> Code

nrg0773-dotcom Create README.md

task3 adding folders

task4 adding folders

task5 adding folders

task6 adding folders

task7 adding folders

README.md Create README.md

Local Codespaces

Clone ?

HTTPS SSH GitHub CLI

https://github.com/nrg0773-dotcom/DSA-Lab1.git

Clone using the web URL.

Open in GitHub Copilot app

Open with GitHub Desktop

Download ZIP

README

Lab 1: Revisiting C++, Introduction to Version Control & Writing Test Cases in C++

Introduction

8. Students will refine .gitignore for C++/Visual Studio

```
# Build artifacts
*.o
*.obj
*.exe
*.pdb
/build/
/Debug/
/Release/

# IDE metadata (keep .sln and .vcxproj if instructed to submit them)
.vscode/
.idea/
```

```
git add .gitignore
git commit -m "Refine .gitignore for C++ build artifacts"
git push
```



9. Confirm History

```
git log --oneline --graph --decorate --all
```

```
branch master set up to track origin/master .
PS D:\DSA\Lab1> git log --oneline --graph --decorate --all
* c932c86 (HEAD -> master, origin/master) addingfolders
PS D:\DSA\Lab1> █
```

Task 2 – Git Workflow Practice (Branch → Implement → PR → Merge)

1. Create and switch to a feature branch

```
git checkout -b lab-tasks
PS D:\DSA\Lab1\task2> git checkout -b lab_tasks
Switched to a new branch 'lab_tasks'
PS D:\DSA\Lab1\task2> git add .
PS D:\DSA\Lab1\task2> git status
On branch lab_tasks
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    new file:   demo.cpp
    new file:   moduleA.cpp
    new file:   moduleA.h
    new file:   test.exe

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    ../.vscode/

PS D:\DSA\Lab1\task2> █
```

2. Add module files

a. Create utilities.h:

```
#ifndef UTILITIES_H
#define UTILITIES_H

int add(int a, int b); // placeholder example

#endif
```

b. Create utilities.cpp:

```
#include "utilities.h"
int add(int a, int b) { return a + b; }
```



3. Add a quick test driver (temporary, will be replaced by full test file later)

File: test_utilities.cpp

```
#include <iostream>
#include "utilities.h"
using namespace std;

int main() {
    cout << "add(2,3) == 5: " << (add(2,3) == 5 ? "PASS" : "FAIL") << endl;
    return 0;
}
```

```
g++ utilities.cpp test_utilities.cpp -o test_utilities
./test_utilities
```

```
PS D:\DSA\Lab1> cd task2
PS D:\DSA\Lab1\task2> g++ demo.cpp moduleA.cpp -o test
```

```
PS D:\DSA\Lab1\task2> ./test.exe
add(2,3) == 5: PASS
PS D:\DSA\Lab1\task2> █
```

4. Stage and commit incrementally

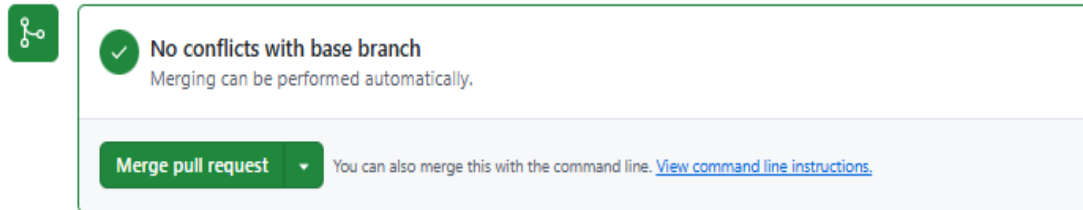
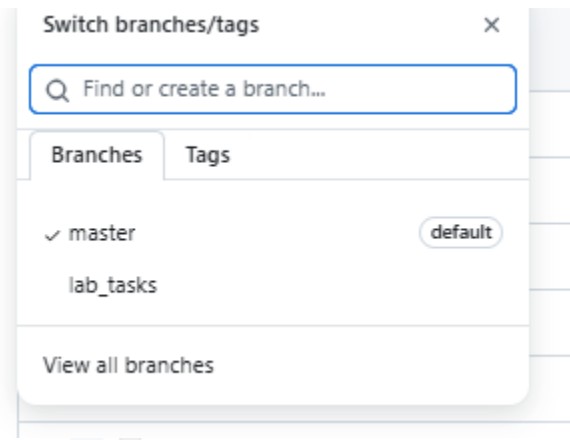
```
git add utilities.h utilities.cpp
git commit -m "Add utilities module with add(a,b)"
git add test_utilities.cpp
git commit -m "Add smoke test for utilities::add"
```

```
● PS D:\DSA\Lab1\task2> git commit -m "added modules"
[lab_tasks 601a7e6] added modules
4 files changed, 29 insertions(+)
create mode 100644 task2/demo.cpp
create mode 100644 task2/moduleA.cpp
create mode 100644 task2/moduleA.h
create mode 100644 task2/test.exe
```



5. Push the branch and create a PR

`git push -u origin lab-tasks`



- On GitHub: Open Pull Request → compare lab-tasks into main.
- Add a clear PR title & description (what changed, why, how tested).
- Request a review (peer or instructor as applicable).
- Make changes locally->commit->push

6. Merge the PR

7. Sync the local main

8. Simulate a simple merge conflict

- On GitHub: edit README.md in main (add a new line).
- Locally on a new branch, edit the same line in README.md, commit, and push.
- Open a PR; GitHub will show a conflict.
- Students will resolve via the GitHub conflict editor or locally:

```
git fetch origin
git checkout lab-tasks
git merge origin/main # triggers conflict locally
# Edit the conflicted files to keep the correct content
git add README.md
```



git commit # completes the merge
git push # PR becomes mergeable

```
# Lab 1: Revisiting C++, Introduction to Version Control & Writing
Test Cases in C++ // altering the ReadMe.md

## Introduction

(Use "git restore <file>..." to discard changes in
modified: ../README.md

PS D:\DSA\Lab1> git log --oneline --decorate --graph --all
>>
* 847f084 (HEAD -> lab_tasks) made changes
* 0b55398 Merge branch 'master' of https://github.com/nrg0773-dotcom/DSA-Lab1 into lab_tasks
|\
| * e7bb0c3 (origin/master) Create README.md
| * 601a7e6 (origin/lab_tasks) added modules
|/
* c932c86 (master) adding folders

moduleA.h x moduleA.cpp x demo.cpp x i REA

task2 > file.txt
1 originla sample code in the master

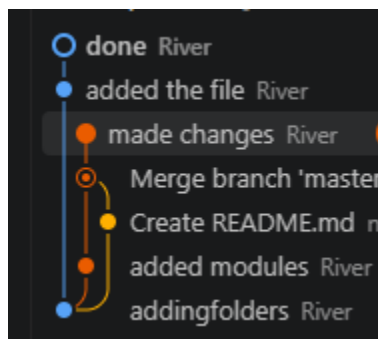
task2 > file.txt
1 originla sample code in the master adding some changes
2 while with in the testbranch

task2 > file.txt
1 originla sample code in the master
2 changing masters after testbranch made its effect
3 from masters

PS D:\DSA\Lab1\task2> git merge master
Auto-merging task2/file.txt
CONFLICT (content): Merge conflict in task2/file.txt
Automatic merge failed; fix conflicts and then commit the result.
PS D:\DSA\Lab1\task2>
```



```
task2 > file.txt
Accept Current Change | Accept Incoming Change | Accept Both Changes | Compare Changes
1 <<<<<<< HEAD (Current Change)
2 originla sample code in the master adding some changes
3 while with in the testbranch
4 =====
5 originla sample code in the master
6 changing masters after testbranch made its effect
7 from masters
8 >>>>>> master (Incoming Change)
9
```



After add + commit both are merged

Task 3 - Find All Indices of an Element

Write a function that finds **all indices** of a given element in an array.

- Input: Array and a key.
- Output: Vector of indices.
- Test cases:
 - Multiple occurrences.
 - Key not present.
 - Empty array.

Task 4 – String Pattern Matching (Naive Algorithm)

Write a function that finds the first occurrence of a substring (pattern) in a given text. If not found, the function will return -1.



- Test cases:
 - Pattern at the beginning.
 - Pattern at the end.
 - Pattern not present.
 - Empty pattern.

Task 5 – Pascal’s Triangle

Write a function that generates Pascal’s Triangle up to n rows.

- Test cases:
 - $n = 0, n = 1, n = 5$.
 - Verify row 5 $\rightarrow \{1, 4, 6, 4, 1\}$.

Task 6 – Histogram Analysis (Mode of Array)

Write a function that returns the **mode** (most frequent element) in an array.

- Test cases:
 - Array with unique mode.
 - Array with multiple modes.
 - Empty array.

Task 7 –Strassen’s Matrix Multiplication

Implement **Strassen’s Algorithm** for multiplying two square matrices (size $2^n \times 2^n$). Compare the result with the standard matrix multiplication algorithm.

- Test cases:
 - 2×2 matrix multiplication.
 - 4×4 matrix multiplication.
 - Random values compared with naive multiplication.

Deliverables

- A single Word document containing:
 - Steps of execution of Task 1 & 2 with screenshots
 - GitHub Repo link (must remain public till final grades are posted)
 - The repo should contain all the source files
- A zip file containing the following for each of Task 3-7
 - A source code file, containing the implementation for the task
 - A source code file, containing the test cases and main method pertaining to the task



National University of Sciences and Technology (NUST) School of Electrical Engineering and Computer Science

- Students must submit both files on LMS before the deadline.
- File naming convention: YourFullName(Reg#)_Lab1

Use proper indentation and comments. Lack of comments and indentation will result in deduction of marks.

Lab Rubrics

Assessment	Doesn't meet Expectation (1–2)	Meets Expectation (3–4)	Exceeds Expectation (5)
Software Problem Realization (CLO2 – PLO2)	The student is unable to outline the problem or use relevant methods.	The student needs guidance to comprehend the problem and identify a suitable method.	The student fully understands the problem, analyzes methods, and develops program flow.
Software Tool Usage (CLO3 – PLO5)	The student shows no command over IDE or Git, code is incomplete or with syntax errors.	The student demonstrates limited command, code compiles but output is inconsistent.	The student demonstrates full command, writes correct and functional code, and manipulates Git tools effectively.
Ethics and Adherence to Lab Rules (CLO4 – PLO8)	The student ignores ethical norms, disrupts lab, careless about safety/punctuality.	The student partially follows ethical norms, shows improved behavior and punctuality.	The student fully adheres to professional ethics, is punctual, and encourages peers to follow lab decorum.